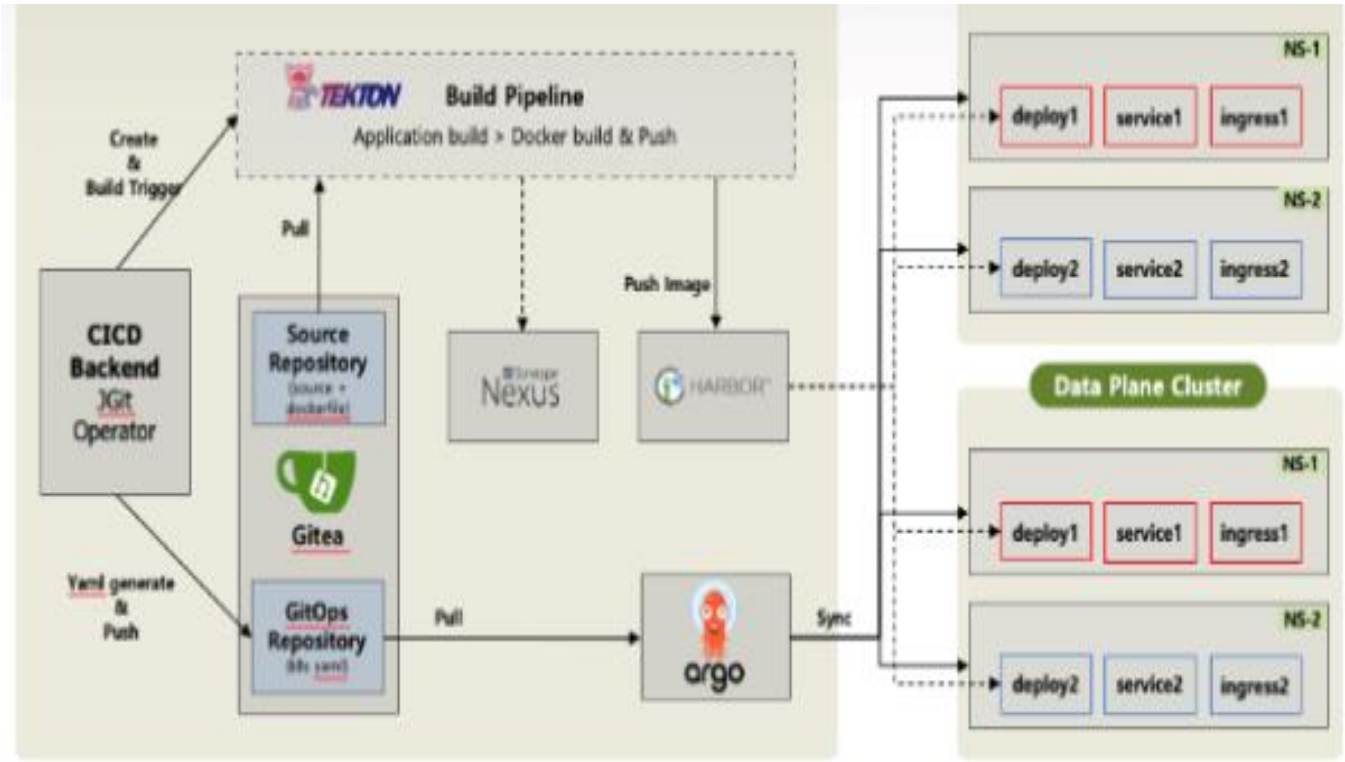
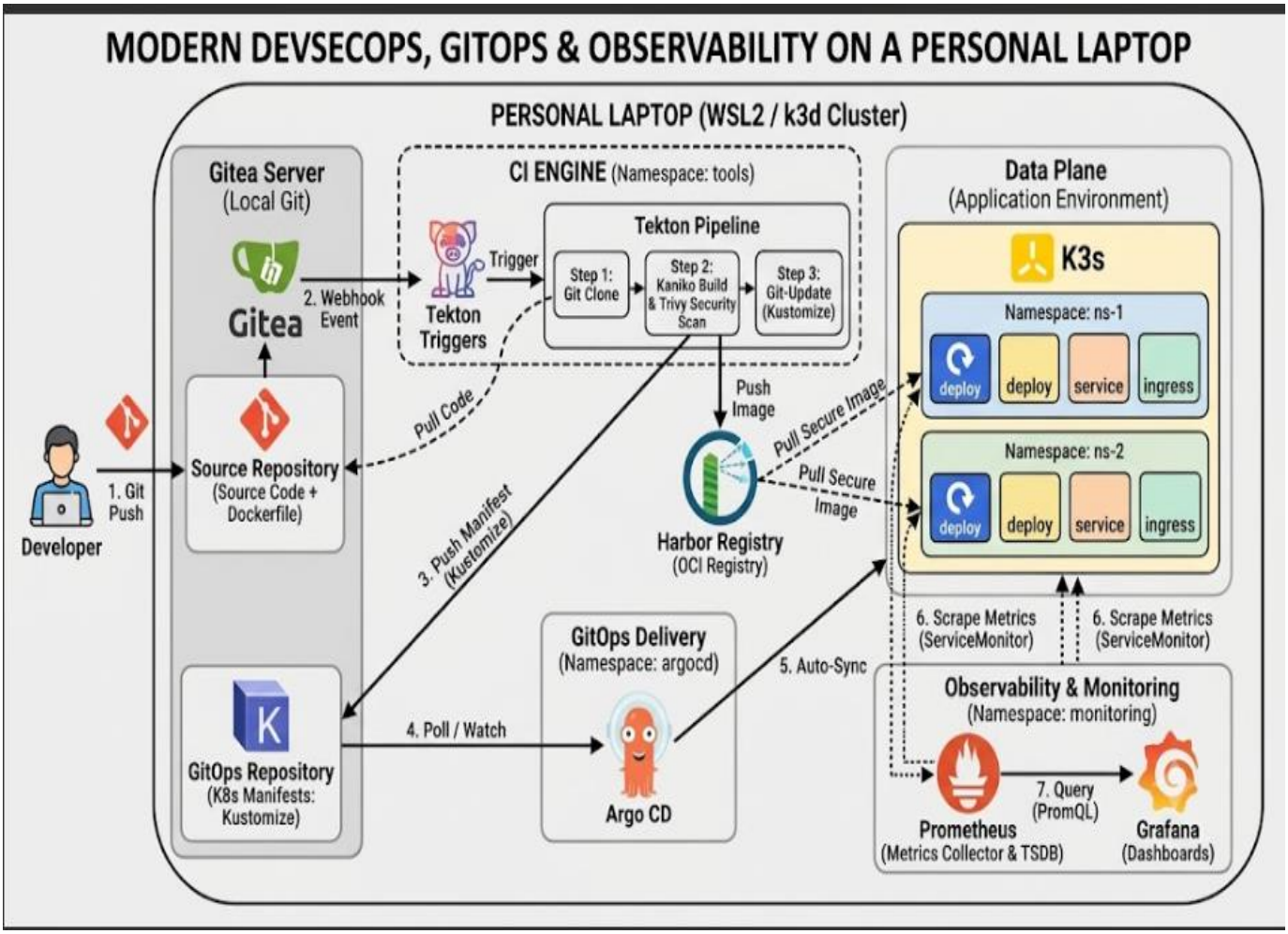

TEKTON
Build Pipeline
Application build > Docker build & Push
Create & Build Trigger
Pull
Push Image
CICD Backend
Xgit Operator
Source Repository
Nexus
HARBOR
Gitea
Yaml generate & Push
GitOps Repository
Pull
argo
Sync
NS-1
deploy1
service1
ingress1
NS-2
deploy2
service2
ingress2
Data Plane Cluster
NS-1
deploy1
service1
ingress1
NS-2
deploy2
service2
ingress2


MODERN DEVSECOPS, GITOPS & OBSERVABILITY ON A PERSONAL LAPTOP
PERSONAL LAPTOP (WSL2 / k3d Cluster)
Gitea Server
(Local Git)
Gitea
2. Webhook Event
CI ENGINE (Namespace: tools)
Tekton Triggers
Trigger
Tekton Pipeline
Step 1: Git Clone
Step 2: Kaniko Build & Trivy Security Scan
Step 3: Git-Update (Kustomize)
Data Plane
(Application Environment)
K3s
Namespace: ns-1
deploy
deploy
service
ingress
Namespace: ns-2
deploy
deploy
service
ingress
Developer
1. Git Push
Source Repository
(Source Code + Dockerfile)
Pull Code
Push Image
Pull Secure Image
Pull Secure Image
Harbor Registry
(OCI Registry)
3. Push Manifest (Kustomize)
GitOps Delivery
(Namespace: argocd)
5. Auto-Sync
6. Scrape Metrics (ServiceMonitor)
6. Scrape Metrics (ServiceMonitor)
Observability & Monitoring
(Namespace: monitoring)
GitOps Repository
(K8s Manifests: Kustomize)
4. Poll / Watch
Argo CD
7. Query (PromQL)
Prometheus
(Metrics Collector & TSDB)
Grafana
(Dashboards)

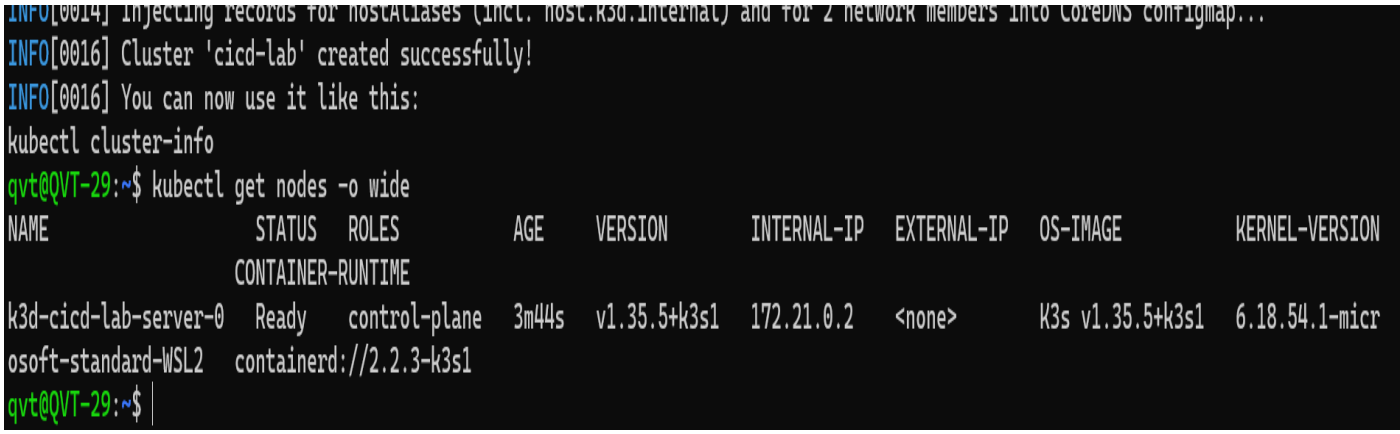
Bảng tổng hợp các điểm bổ sung và thay thế cụ thể

Thành phần / Tính năng	Sơ đồ cũ	Sơ đồ mới	Bản chất cải tiến
Quét bảo mật (Security Scan)	Không có	Tích hợp Trivy Scan	Bổ sung mới: Phát hiện lỗ hổng image trước khi push lên Harbor.
Cơ chế kích hoạt CI	CICD Backend Operator	Tekton Triggers (Webhook)	Thay thế: Tận dụng công cụ native của Tekton, loại bỏ code trung gian.
Cơ chế đóng gói Image	Docker build thông thường	Kaniko Engine	Bổ sung mới: Build image an toàn bên trong Kubernetes Pod không cần Docker socket.
Quản lý Artifact/Package	Sonatype Nexus + Harbor	Chỉ dùng Harbor Registry	Cắt giảm tối ưu: Giảm tải 2–4 GB RAM, tập trung hóa quản lý OCI image.
Cập nhật Manifest GitOps	JGit Operator tự render YAML	Task Git-Update qua Kustomize	Thay thế: Cập nhật tag image chuẩn GitOps khai báo, hỗ trợ multi-environment gọn gàng.
Môi trường triển khai	Data Plane Cluster tách rời	K3s/k3d phân vùng bằng Namespaces (ns-1, ns-2)	Tối ưu hóa Local: Chạy trọn vẹn toàn bộ kiến trúc trên một laptop cá nhân.
Giám sát & Trực quan hóa (Observability & Monitoring)	Không có (Hệ thống chạy mù mờ, không có công cụ đo đạc số liệu)	Tích hợp Prometheus & Grafana (Namespace: monitoring)	Bổ sung mới: Hoàn thiện vòng đời khép kín (GitOps & Observability). Tự động cào và trực quan hóa thời gian thực 3 tầng metrics: Dây chuyền CI/CD (Tekton + Argo CD), tài nguyên cụm K8s và ứng dụng Data Plane.

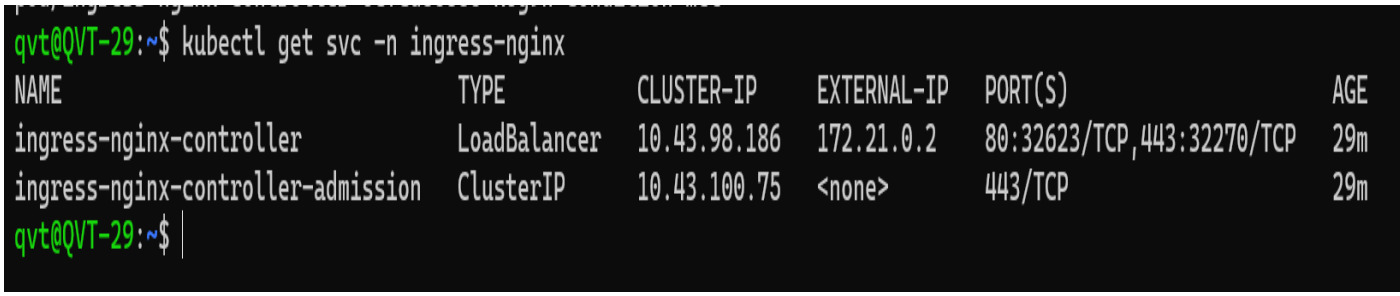
Giai đoạn 1: Khởi tạo Cụm K3d và Phân vùng Không gian tên (Namespaces)

- **Mục tiêu:** Dựng cụm Kubernetes nội bộ trên WSL2 và tạo phân vùng mạng logic cho các thành phần.
- **Các bước thực hiện:**

1. **Khởi tạo cụm K3d:** Tạo cluster với tên cicd-lab, mở port 80/443 để phục vụ Ingress traffic và vô hiệu hóa Traefik mặc định (nếu muốn dùng Ingress-Nginx):



2. **Cài đặt Ingress Controller:** Cài đặt Ingress-Nginx Controller để quản lý định tuyến HTTP/HTTPS cho Gitea, Harbor, Argo CD và các ứng dụng Data Plane.



3. **Khởi tạo các Namespaces chuyên biệt:**
    - tools: Chứa toàn bộ hệ thống lõi CI (Gitea, Tekton Pipelines, Harbor).
    - argocd: Chứa controller và giao diện của Argo CD.
    - ns-1 & ns-2: Môi trường Data Plane thực thi ứng dụng.

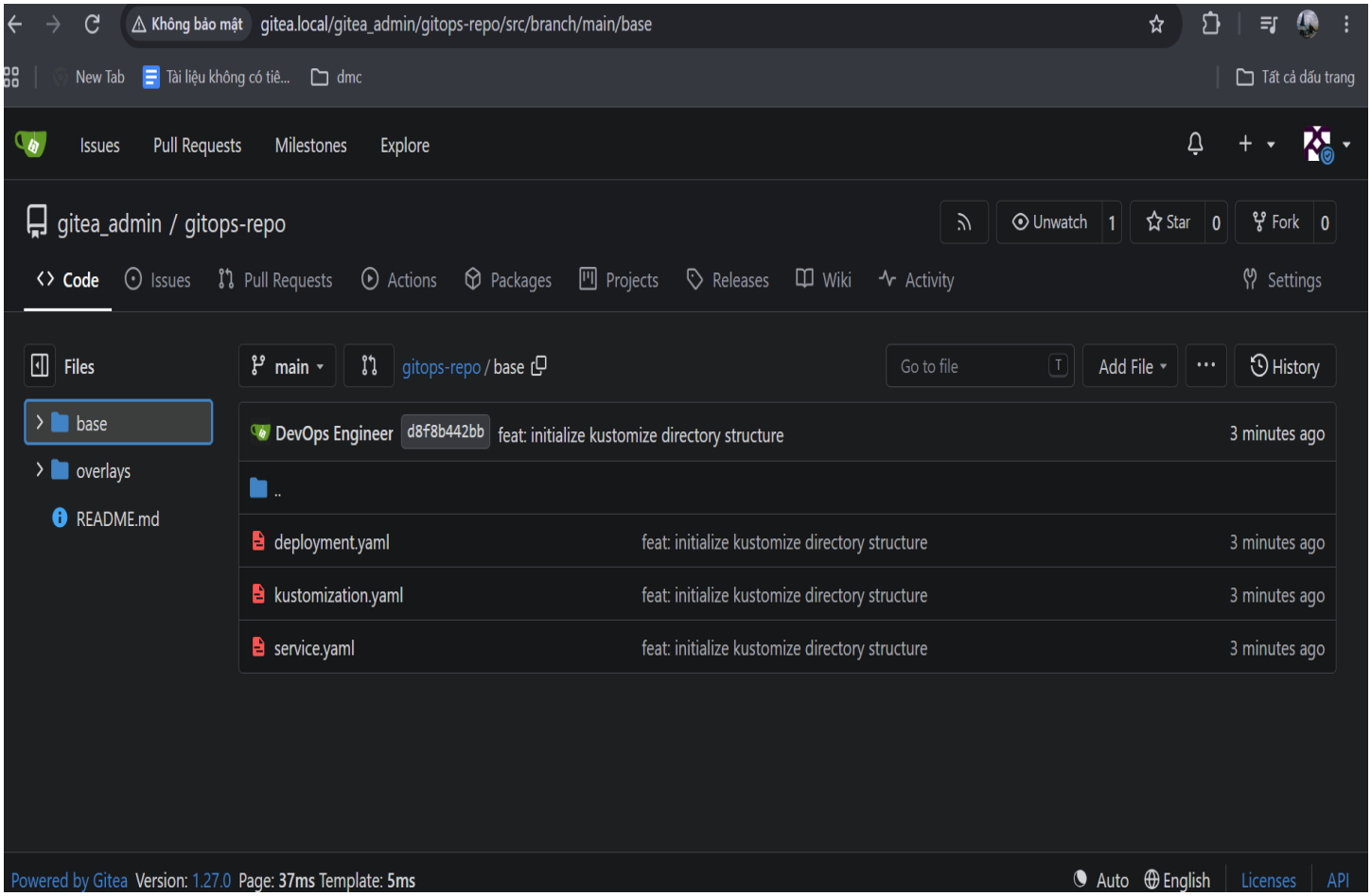
```
=== 2. KIỂM TRA NAMESPACES ===
NAME              STATUS   AGE     LABELS
argocd            Active   7m6s    env=gitops-controller,kubernetes.io/metadata.name=argocd
default           Active   8h      kubernetes.io/metadata.name=default
ingress-nginx     Active   91m     kubernetes.io/metadata.name=ingress-nginx,name=ingress-nginx
kube-node-lease   Active   8h      kubernetes.io/metadata.name=kube-node-lease
kube-public       Active   8h      kubernetes.io/metadata.name=kube-public
kube-system       Active   8h      kubernetes.io/metadata.name=kube-system
ns-1              Active   7m2s    env=dataplane-ns1,kubernetes.io/metadata.name=ns-1
ns-2              Active   6m58s   env=dataplane-ns2,kubernetes.io/metadata.name=ns-2
tools             Active   7m11s   env=infrastructure-tools,kubernetes.io/metadata.name=tools

=== 3. KIỂM TRA INGRESS CONTROLLER ===
```

Giai đoạn 2: Cài đặt Dịch vụ Mã nguồn và Quản lý Image (Gitea & Harbor)

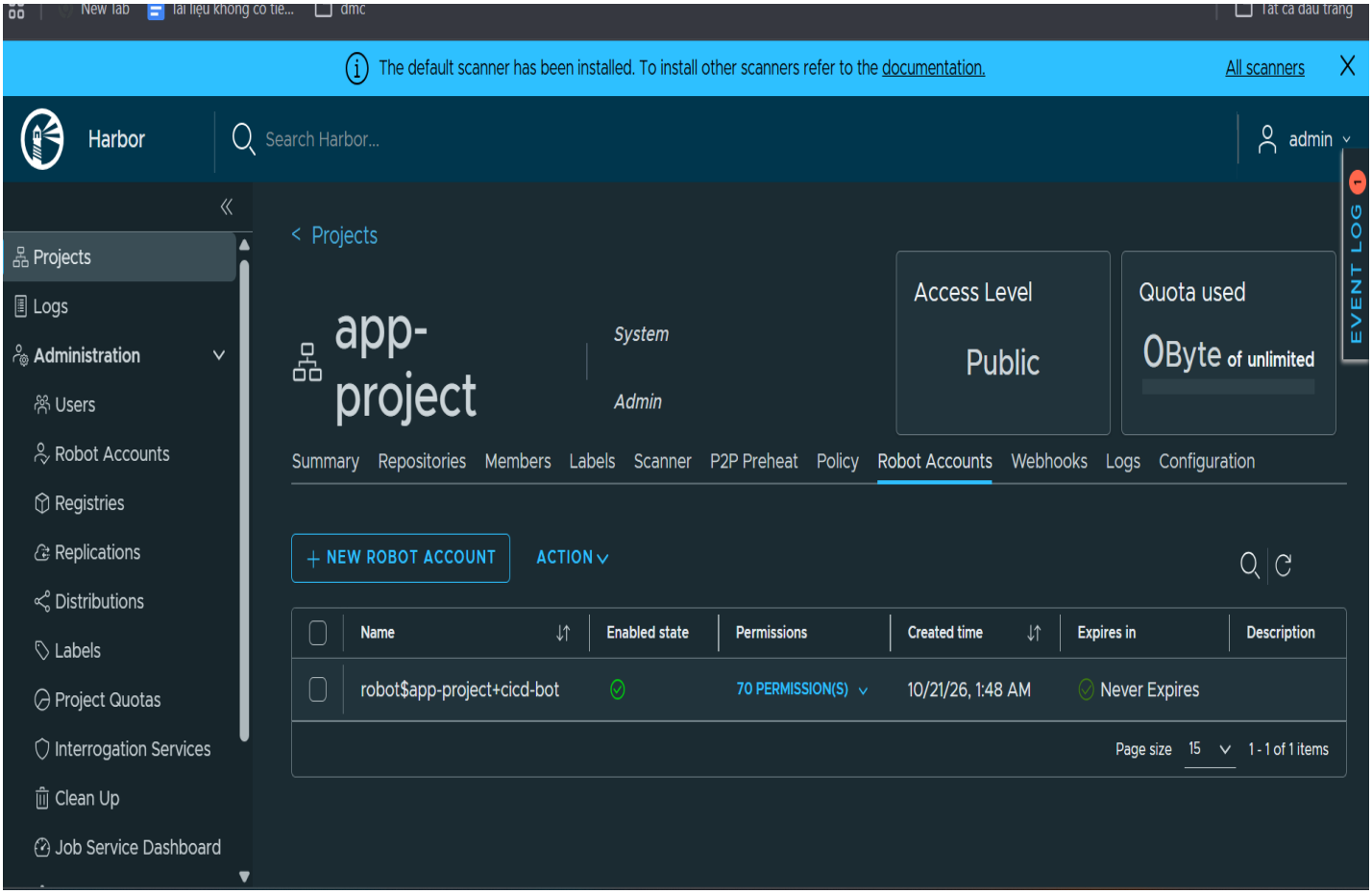
- **Mục tiêu:** Thiết lập kho lưu trữ mã nguồn cục bộ và container registry có tính năng bảo mật.
- **Các bước thực hiện:**

1. **Triển khai Gitea:**
   - Cài đặt Gitea vào namespace tools qua Helm Chart, cấu hình SQLite làm cơ sở dữ liệu để tối ưu RAM.
   - Cấu hình Ingress domain nội bộ (ví dụ: http://gitea.local).
   - Đăng nhập tài khoản admin, khởi tạo 2 repository:
   - source-repo: Chứa mã nguồn ứng dụng + Dockerfile.
   - gitops-repo: Chứa các file manifest Kubernetes cấu trúc theo Kustomize (base/, overlays/ns-1/, overlays/ns-2/).

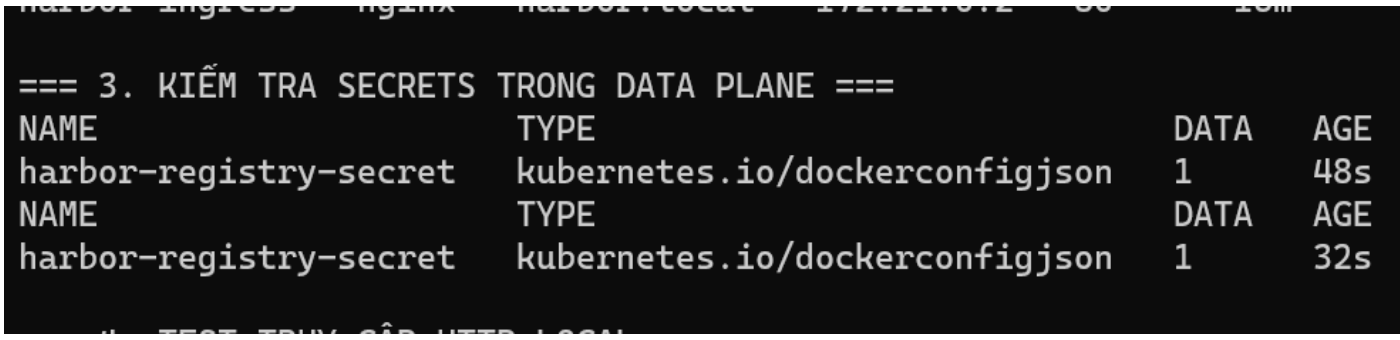


2. **Triển khai Harbor OCI Registry:**

- Cài đặt Harbor qua Helm Chart vào namespace tools (tắt Notary, bật hoặc tắt Trivy nội bộ tùy thuộc dung lượng RAM).
- Tạo một Project dạng Private/Public tên là app-project.
- Tạo một Robot Account có quyền push/pull image.

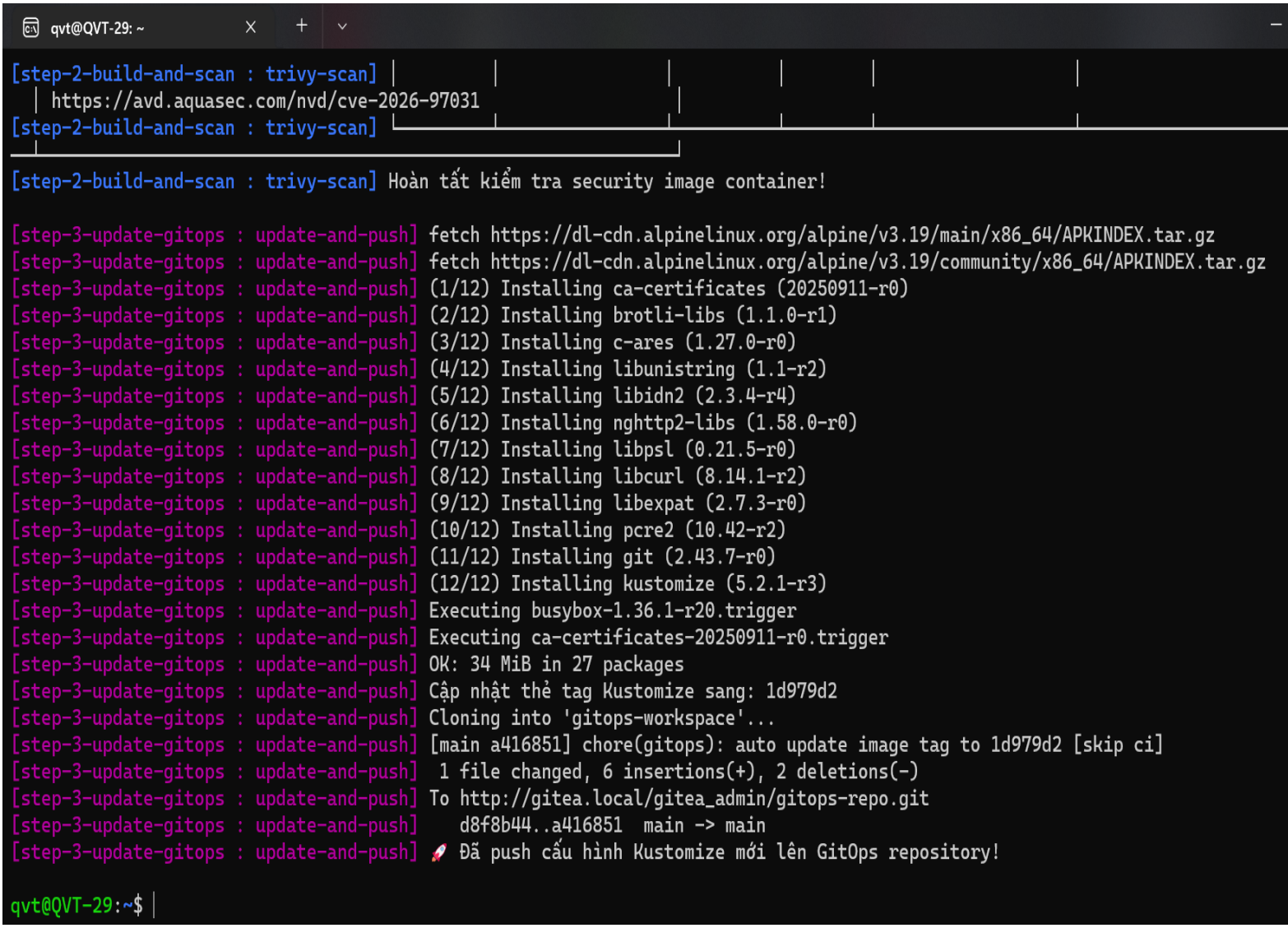


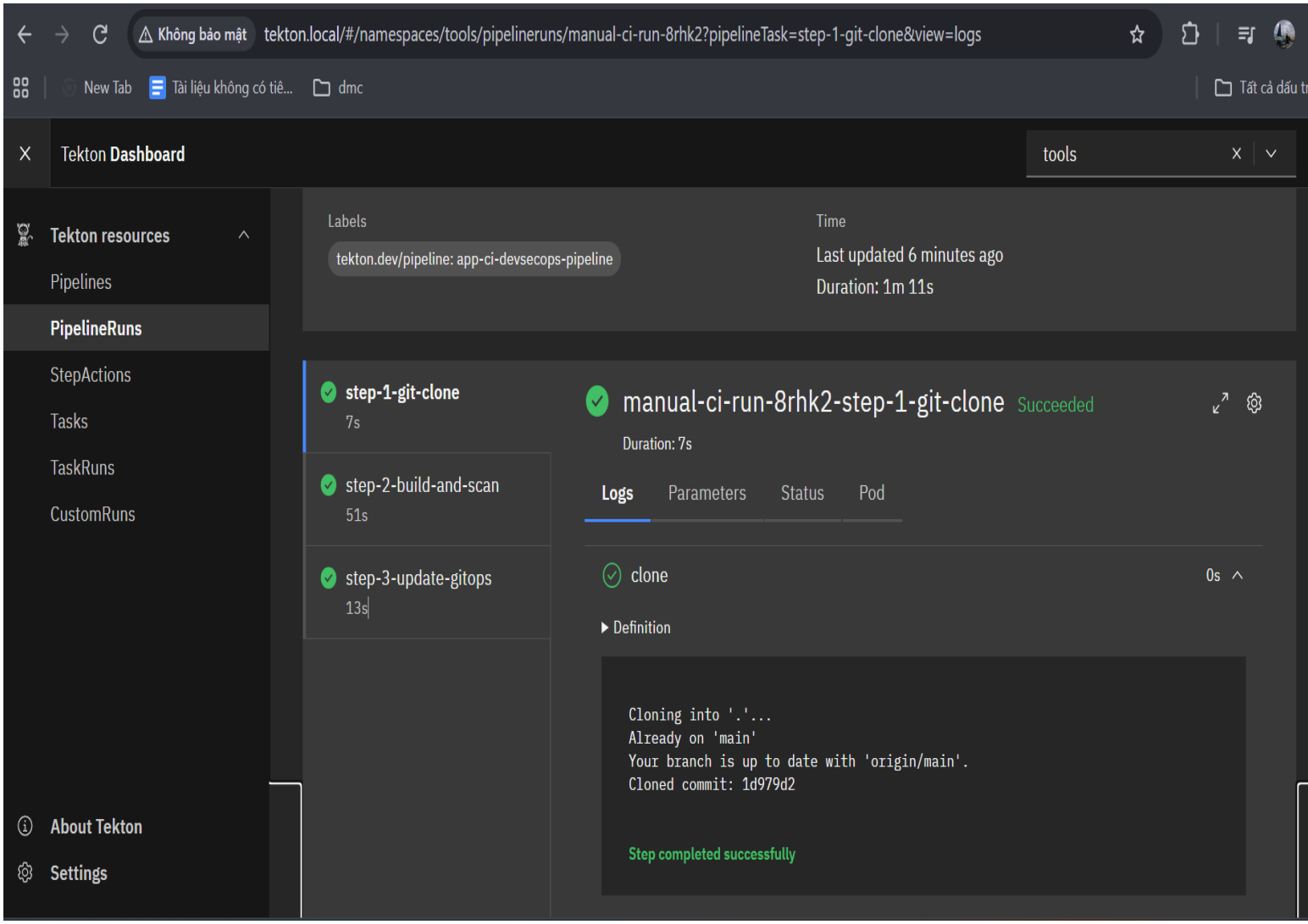
3. **Phân phối Image Pull Secret:** Tạo Secret kiểu docker-registry chứa thông tin đăng nhập của Harbor và đồng bộ sang các namespace ns-1 và ns-2 để cụm K8s kéo được image về chạy.

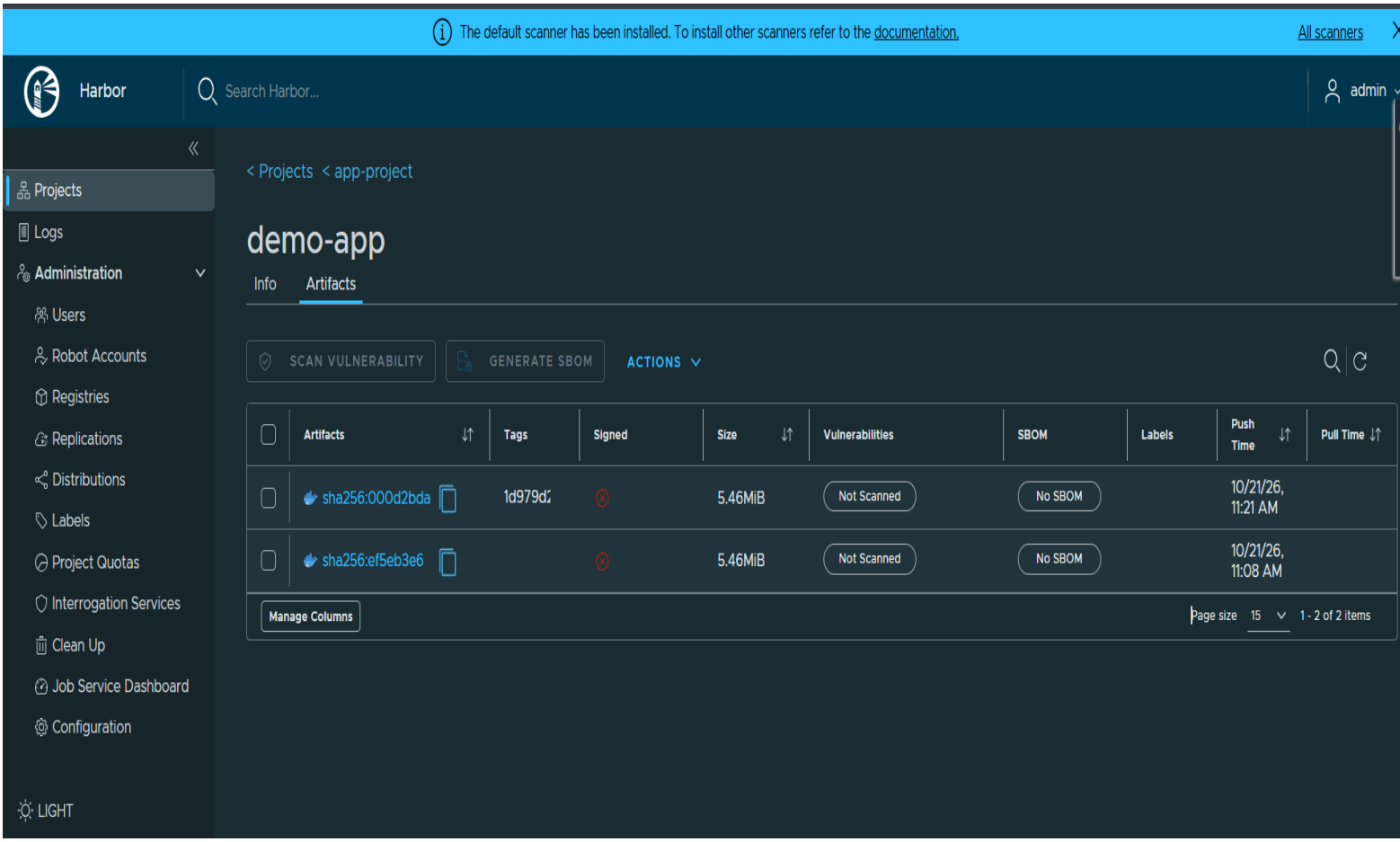


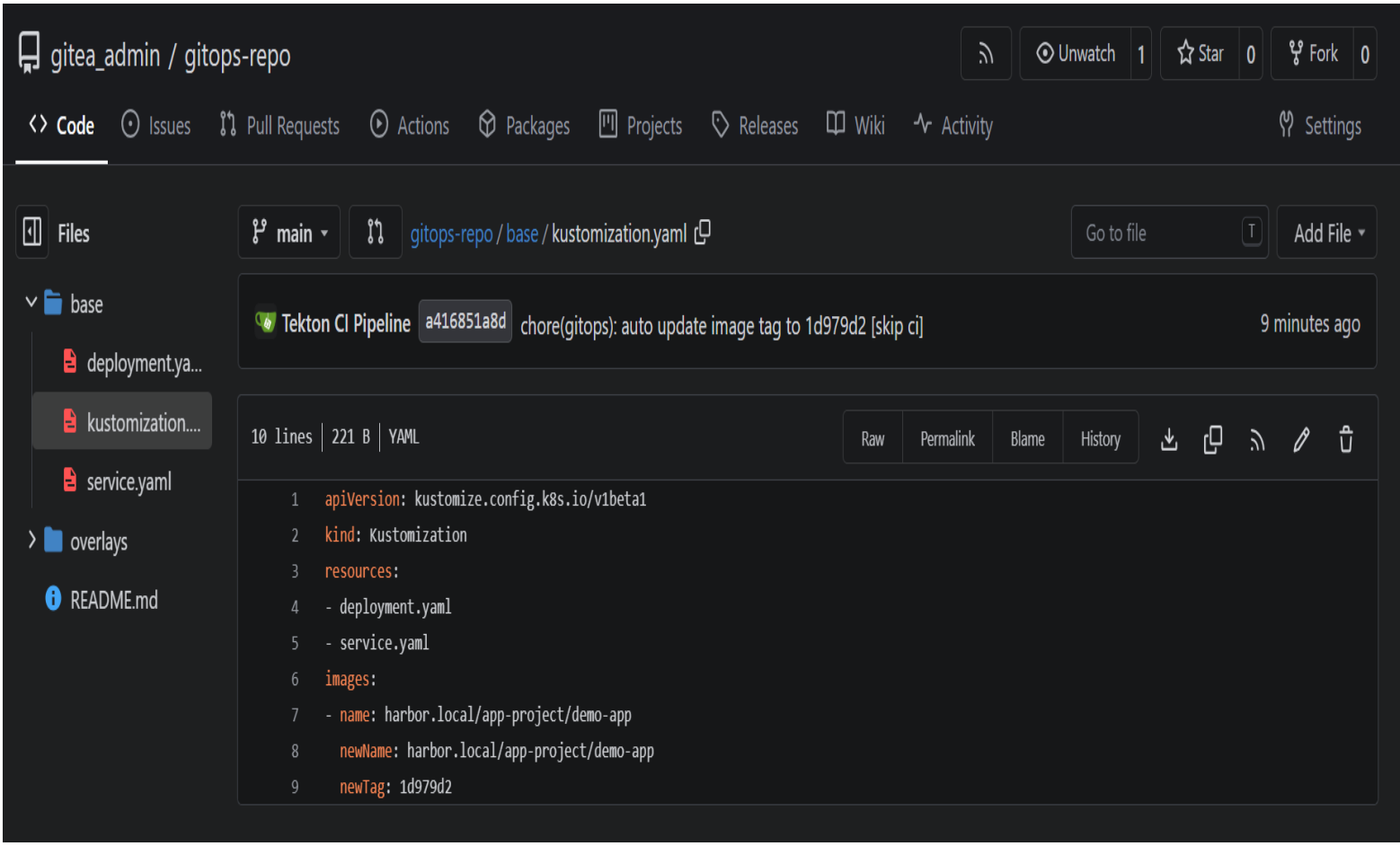
Giai đoạn 3: Xây dựng Khối CI Native và Quét bảo mật (Tekton Pipeline)

- **Mục tiêu:** Đóng gói ứng dụng mẫu, quét lỗ hổng container và tự động cập nhật thẻ tag vào GitOps.
- **Các bước thực hiện:**
    1. **Cài đặt Tekton Core & Dashboard:** Triển khai Tekton Pipelines và Tekton Dashboard vào namespace tekton-pipelines (hoặc tools).
    2. **Chuẩn bị ứng dụng mẫu:**
        - Viết một ứng dụng web nhẹ (Golang, Python FastAPI hoặc Node.js) trả về chuỗi text và phiên bản (v1).
        - Đẩy toàn bộ code và Dockerfile lên source-repo của Gitea.
    3. **Xây dựng các Tekton Tasks:**
        - **Step 1: Git Clone Task:** Sử dụng task chuẩn từ Tekton Hub để clone commit mới nhất từ source-repo.
        - **Step 2: Kaniko Build & Trivy Scan Task:**
            - Kaniko xây dựng Docker image từ Dockerfile mà không cần Docker daemon.
            - Tích hợp Trivy CLI để quét lỗ hổng (CVE) của image vừa build. Nếu phát hiện lỗi nghiêm trọng (Critical/High), pipeline có thể dừng lại hoặc ghi cảnh báo.
            - Đẩy image hoàn chỉnh lên Harbor với tag tương ứng mã Git Commit SHA: harbor.local/app-project/demo-app:<git-commit-sha>.
        - **Step 3: Git-Update (Kustomize) Task:**
            - Clone kho gitops-repo.
            - Sử dụng lệnh kustomize edit set image để thay đổi tag image trong file cấu hình sang <git-commit-sha> mới nhất.
            - Thực hiện lệnh git commit -m "chore: update image to <git-commit-sha>" và git push trở lại Gitea gitops-repo.
    4. **Kiểm thử Pipeline thủ công:** Chạy thử nghiệm bằng một PipelineRun thủ công để đảm bảo luồng build -> scan -> push image -> push commit GitOps hoạt động thông suốt.


qvt@QVT-29: ~
[step-2-build-and-scan : trivy-scan] | | | | | | |
| https://avd.aquasec.com/nvd/cve-2026-97031 |
[step-2-build-and-scan : trivy-scan] └──────┴──────┴──────┴──────┴──────┴──────┴──────┘
[step-2-build-and-scan : trivy-scan] Hoàn tất kiểm tra security image container!
[step-3-update-gitops : update-and-push] fetch https://dl-cdn.alpinelinux.org/alpine/v3.19/main/x86_64/APKINDEX.tar.gz
[step-3-update-gitops : update-and-push] fetch https://dl-cdn.alpinelinux.org/alpine/v3.19/community/x86_64/APKINDEX.tar.gz
[step-3-update-gitops : update-and-push] (1/12) Installing ca-certificates (20250911-r0)
[step-3-update-gitops : update-and-push] (2/12) Installing brotli-libs (1.1.0-r1)
[step-3-update-gitops : update-and-push] (3/12) Installing c-ares (1.27.0-r0)
[step-3-update-gitops : update-and-push] (4/12) Installing libunistring (1.1-r2)
[step-3-update-gitops : update-and-push] (5/12) Installing libidn2 (2.3.4-r4)
[step-3-update-gitops : update-and-push] (6/12) Installing nghttp2-libs (1.58.0-r0)
[step-3-update-gitops : update-and-push] (7/12) Installing libpsl (0.21.5-r0)
[step-3-update-gitops : update-and-push] (8/12) Installing libcurl (8.14.1-r2)
[step-3-update-gitops : update-and-push] (9/12) Installing libexpat (2.7.3-r0)
[step-3-update-gitops : update-and-push] (10/12) Installing pcre2 (10.42-r2)
[step-3-update-gitops : update-and-push] (11/12) Installing git (2.43.7-r0)
[step-3-update-gitops : update-and-push] (12/12) Installing kustomize (5.2.1-r3)
[step-3-update-gitops : update-and-push] Executing busybox-1.36.1-r20.trigger
[step-3-update-gitops : update-and-push] Executing ca-certificates-20250911-r0.trigger
[step-3-update-gitops : update-and-push] OK: 34 MiB in 27 packages
[step-3-update-gitops : update-and-push] Cập nhật thẻ tag Kustomize sang: 1d979d2
[step-3-update-gitops : update-and-push] Cloning into 'gitops-workspace'...
[step-3-update-gitops : update-and-push] [main a416851] chore(gitops): auto update image tag to 1d979d2 [skip ci]
[step-3-update-gitops : update-and-push] 1 file changed, 6 insertions(+), 2 deletions(-)
[step-3-update-gitops : update-and-push] To http://gitea.local/gitea_admin/gitops-repo.git
[step-3-update-gitops : update-and-push] d8f8b44..a416851 main -> main
[step-3-update-gitops : update-and-push] 🚀 Đã push cấu hình Kustomize mới lên GitOps repository!
qvt@QVT-29:~$


Không bảo mật
tekton.local/#/namespaces/tools/pipelineruns/manual-ci-run-8rhk2?pipelineTask=step-1-git-clone&view=logs
New Tab
Tài liệu không có tiê...
dmc
Tất cả dấu tr
Tekton Dashboard
tools
Tekton resources
Pipelines
PipelineRuns
StepActions
Tasks
TaskRuns
CustomRuns
About Tekton
Settings
Labels
tekton.dev/pipeline: app-ci-devsecops-pipeline
Time
Last updated 6 minutes ago
Duration: 1m 11s
step-1-git-clone
7s
step-2-build-and-scan
51s
step-3-update-gitops
13s
manual-ci-run-8rhk2-step-1-git-clone Succeeded
Duration: 7s
Logs
Parameters
Status
Pod
clone
0s
Definition
Cloning into '.'...
Already on 'main'
Your branch is up to date with 'origin/main'.
Cloned commit: 1d979d2
Step completed successfully


The default scanner has been installed. To install other scanners refer to the documentation.
All scanners
Harbor
Search Harbor...
admin
Projects
Logs
Administration
Users
Robot Accounts
Registries
Replications
Distributions
Labels
Project Quotas
Interrogation Services
Clean Up
Job Service Dashboard
Configuration
LIGHT
< Projects < app-project
demo-app
Info
Artifacts
SCAN VULNERABILITY
GENERATE SBOM
ACTIONS
Artifacts
Tags
Signed
Size
Vulnerabilities
SBOM
Labels
Push Time
Pull Time
sha256:000d2bda
1d979d2
5.46MiB
Not Scanned
No SBOM
10/21/26, 11:21 AM
sha256:ef5eb3e6
5.46MiB
Not Scanned
No SBOM
10/21/26, 11:08 AM
Manage Columns
Page size 15
1 - 2 of 2 items

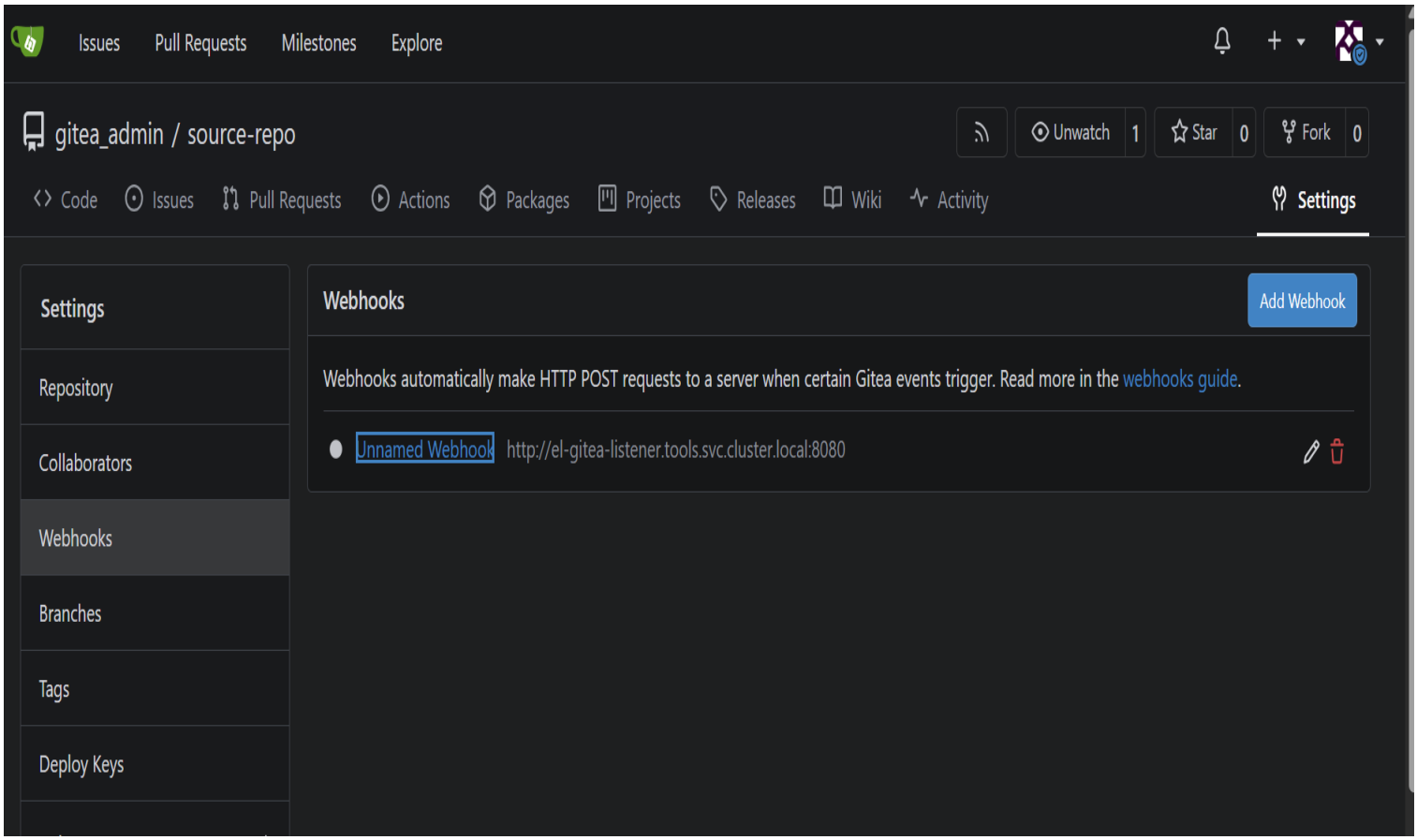


Giai đoạn 4: Kết nối Tự động hóa Trigger (Tekton Triggers)

- **Mục tiêu:** Thay thế hoàn toàn Java Operator bằng cơ chế Webhook-to-Pipeline chuẩn Cloud-Native.
- **Các bước thực hiện:**
    1. **Triển khai Tekton Triggers:** Cài đặt bộ điều khiển Triggers vào cụm.
    2. **Định nghĩa các tài nguyên Trigger:**
        - TriggerBinding: Trích xuất thông tin commit (Git URL, Commit SHA, Branch name) từ JSON payload mà Gitea gửi sang.
        - TriggerTemplate: Bản thiết kế định nghĩa cách tạo ra một PipelineRun tương ứng với các biến lấy từ binding.
        - EventListener: Một Pod lắng nghe HTTP request từ bên ngoài để kích hoạt TriggerTemplate.

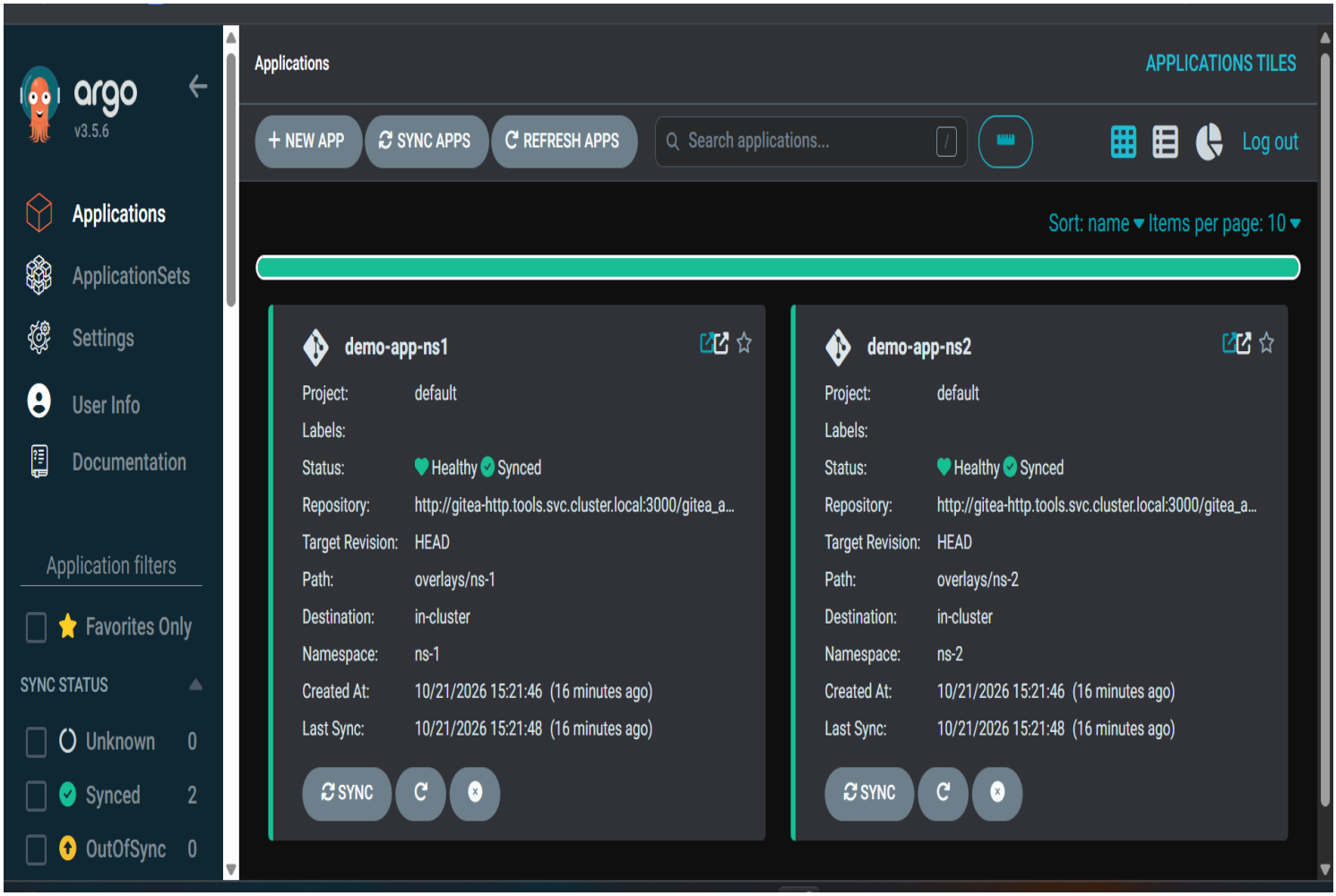
```
qvt@QVT-29:~$ kubectl get pods -n tools -l eventlistener=gitea-listene
NAME                                  READY   STATUS    RESTARTS   AGE
el-gitea-listener-7bc9cf7b95-585wl    1/1     Running   0          18s
```

3. **Cấu hình Webhook trên Gitea:**
    - Tạo Ingress trỏ đến dịch vụ của EventListener.
    - Trong source-repo của Gitea, thêm một Webhook trỏ tới đường dẫn EventListener, kích hoạt tại sự kiện Push Events.



Giai đoạn 5: Cấu hình Khối GitOps Delivery (Argo CD)

- **Mục tiêu:** Đồng bộ tự động mã khai báo từ GitOps Repo sang các namespace Data Plane (ns-1, ns-2).
- **Các bước thực hiện:**
    1. **Triển khai Argo CD:** Cài đặt bản chính thức vào namespace argocd, cấu hình truy cập giao diện qua Ingress hoặc Port-forward.
    2. **Kết nối GitOps Repository:** Khai báo thông tin xác thực để Argo CD có quyền đọc kho gitops-repo trên Gitea.
    3. **Định nghĩa cấu hình triển khai Kustomize:**
        - Thư mục base/: Chứa file deployment.yaml, service.yaml, ingress.yaml cơ sở.
        - Thư mục overlays/ns-1/: Cấu hình riêng cho namespace ns-1 (tên miền ingress, số lượng replica).
        - Thư mục overlays/ns-2/: Cấu hình riêng cho namespace ns-2.
    4. **Tạo Argo CD Applications:**
        - Tạo đối tượng Application số 1: Trỏ tới đường dẫn overlays/ns-1, đích đến là cụm local namespace ns-1.
        - Tạo đối tượng Application số 2: Trỏ tới đường dẫn overlays/ns-2, đích đến là cụm local namespace ns-2.
        - Bật các chính sách đồng bộ: Automated Sync Policy, Self-Heal (tự sửa đổi nếu cluster bị chỉnh tay) và Prune (tự xóa tài nguyên thừa).



Giai đoạn 6: Kiểm thử Toàn trình (End-to-End Test) và Đánh giá

- **Mục tiêu:** Xác thực tính khép kín và tự động hóa 100% của toàn bộ luồng từ máy lập trình viên đến môi trường ứng dụng.
- **Các bước thực hiện:**
    1. **Mô phỏng chu trình làm việc:**
        - Lập trình viên sửa file mã nguồn ứng dụng trên máy tính cá nhân (ví dụ: đổi text phản hồi từ "Hello v1" thành "Hello v2 - Modern DevSecOps").
        - Chạy lệnh git add, git commit và git push lên source-repo của Gitea.
    2. **Theo dõi chuỗi phản ứng:**
        - Gitea phát tín hiệu Webhook -> Tekton Triggers kích hoạt Tekton Pipeline.
        - Tekton tự động kéo code, Kaniko build image, Trivy quét lỗ hổng và đẩy image lên Harbor.

- Tekton cập nhật tag image mới vào gitops-repo.
- Argo CD phát hiện commit mới từ gitops-repo, chuyển trạng thái sang OutOfSync và tự động thực hiện quá trình Sync xuống các namespace ns-1 và ns-2.
- Các Pod trong ns-1 và ns-2 tiến hành cập nhật dạng Rolling Update, kéo image mới trực tiếp từ Harbor về.

```
qvt@QVT-29:~$ echo "=== Image trên NS-1 ==="
kubectl get deployment demo-app -n ns-1 -o jsonpath='{.spec.template.spec.containers[0].im

echo "=== Image trên NS-2 ==="
kubectl get deployment demo-app -n ns-2 -o jsonpath='{.spec.template.spec.containers[0].im
=== Image trên NS-1 ===
harbor.local/app-project/demo-app:409af91
=== Image trên NS-2 ===
harbor.local/app-project/demo-app:409af91
```

3. **Xác thực kết quả :**
   - Gửi request curl tới Ingress của ứng dụng tại ns-1 và ns-2 để kiểm tra chuỗi text mới đã hiển thị.

```
qvt@QVT-29:~$ echo "--- Kiểm tra Endpoint NS-1 ---"
curl -s http://app-ns1.local
--- Kiểm tra Endpoint NS-1 ---
🚀 Xin chào từ Data Plane! Phiên bản ứng dụng: v2.0.0 - Modern Dev
qvt@QVT-29:~$ echo -e "\n--- Kiểm tra Endpoint NS-2 ---"
curl -s http://app-ns2.local

--- Kiểm tra Endpoint NS-2 ---
🚀 Xin chào từ Data Plane! Phiên bản ứng dụng: v2.0.0 - Modern Dev
qvt@QVT-29:~$
```

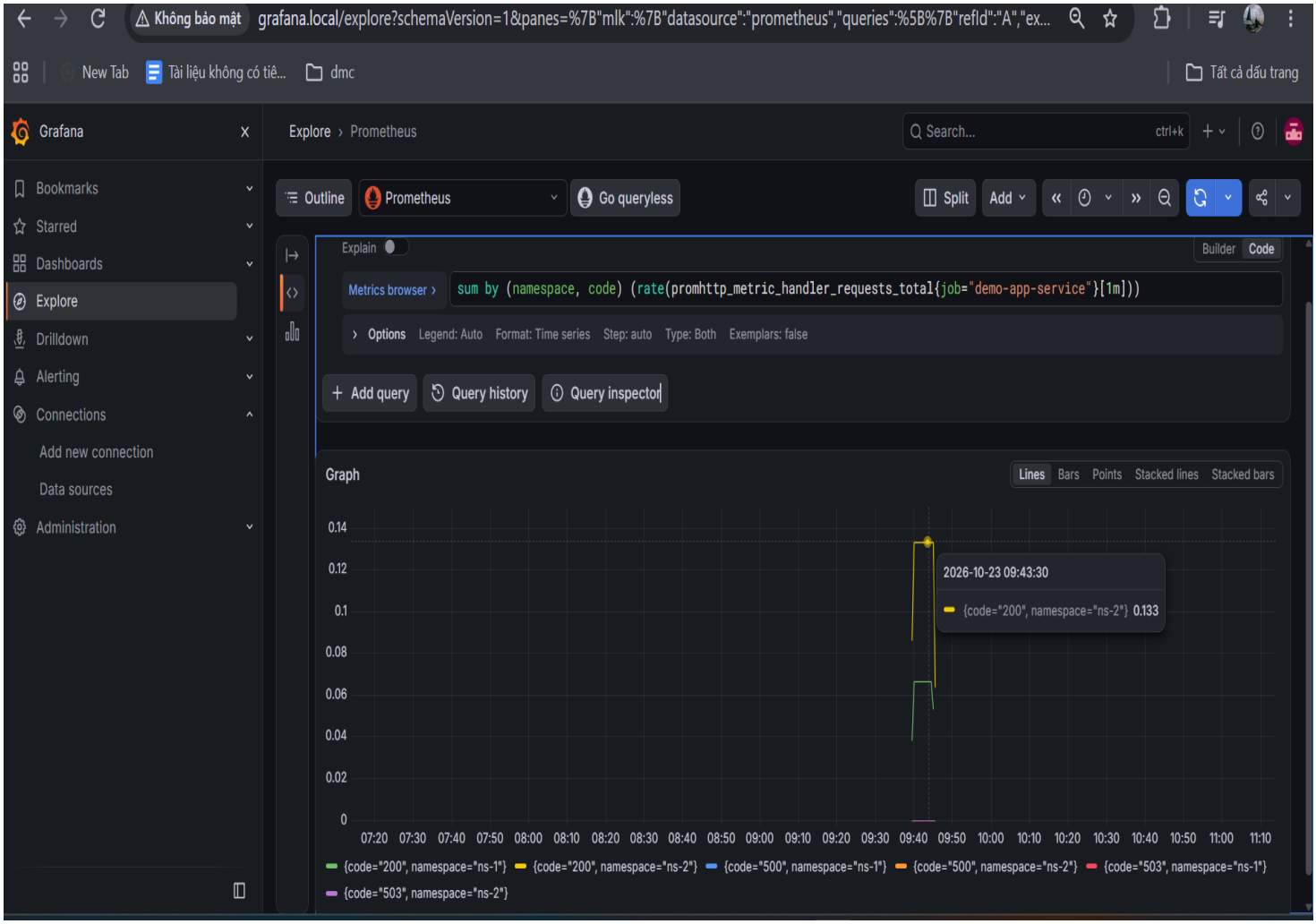
- Đo đạc tài nguyên tiêu thụ bằng lệnh kubectl top nodes và kubectl top pods - A để đảm bảo hệ thống duy trì độ ổn định trên laptop.

```
qvt@QVT-29:~$ kubectl top nodes
NAME                   CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
k3d-cicd-lab-server-0   482m         6%       2971Mi          38%
qvt@QVT-29:~$ kubectl top pods -A --sort-by=memory
NAMESPACE                   NAME                                                CPU(cores)   MEMORY(bytes)
tools                       gitea-postgresql-ha-pgpool-76c988848d-j8rmc          26m          213Mi
tools                       gitea-postgresql-ha-postgresql-0                    57m          161Mi
ingress-nginx               ingress-nginx-controller-6c7cd85885-pfz6x           3m           134Mi
tools                       gitea-postgresql-ha-postgresql-2                    32m          106Mi
tools                       gitea-54887b78ff-fplgh                              6m           97Mi
argocd                      argocd-application-controller-0                     6m           97Mi
tools                       gitea-postgresql-ha-postgresql-1                    42m          88Mi
tools                       harbor-database-0                                   9m           78Mi
tools                       harbor-core-5bcc7c678d-5cc28                        2m           58Mi
tekton-pipelines            tekton-pipelines-webhook-75cd84877-h4jpb            2m           49Mi
argocd                      argocd-dex-server-776dbb476d-hmb2q                  1m           46Mi
tekton-pipelines            tekton-pipelines-controller-65f567589b-cs4v5        2m           46Mi
argocd                      argocd-repo-server-58c976b9df-rcsn4                 1m           39Mi
argocd                      argocd-server-7fd984ff9-5tmms                       1m           36Mi
kube-system                 metrics-server-786d997795-tpkjj                     6m           36Mi
kube-system                 coredns-8db54c48d-7glgk                             5m           33Mi
tools                       harbor-registry-647f895d76-m8fqn                    1m           32Mi
tekton-pipelines-resolvers  tekton-pipelines-remote-resolvers-59b7b847cd-v7r5k   3m           30Mi
tekton-pipelines            tekton-triggers-controller-66fd74568d-l88pp         2m           27Mi
tools                       harbor-trivy-0                                      1m           26Mi
argocd                      argocd-applicationset-controller-7567dc5fbb-qj2lp   1m           26Mi
tekton-pipelines            tekton-triggers-webhook-55c8dd895f-2mhb6            2m           24Mi
tekton-pipelines            tekton-events-controller-5cbc777ccd-gr92v           2m           24Mi
tekton-pipelines            tekton-triggers-core-interceptors-66456f8cf6-trm5s  1m           21Mi
tools                       el-gitea-listener-7bc9cf7b95-585wl                  1m           18Mi
```

Giai đoạn 7: Observability & Monitoring

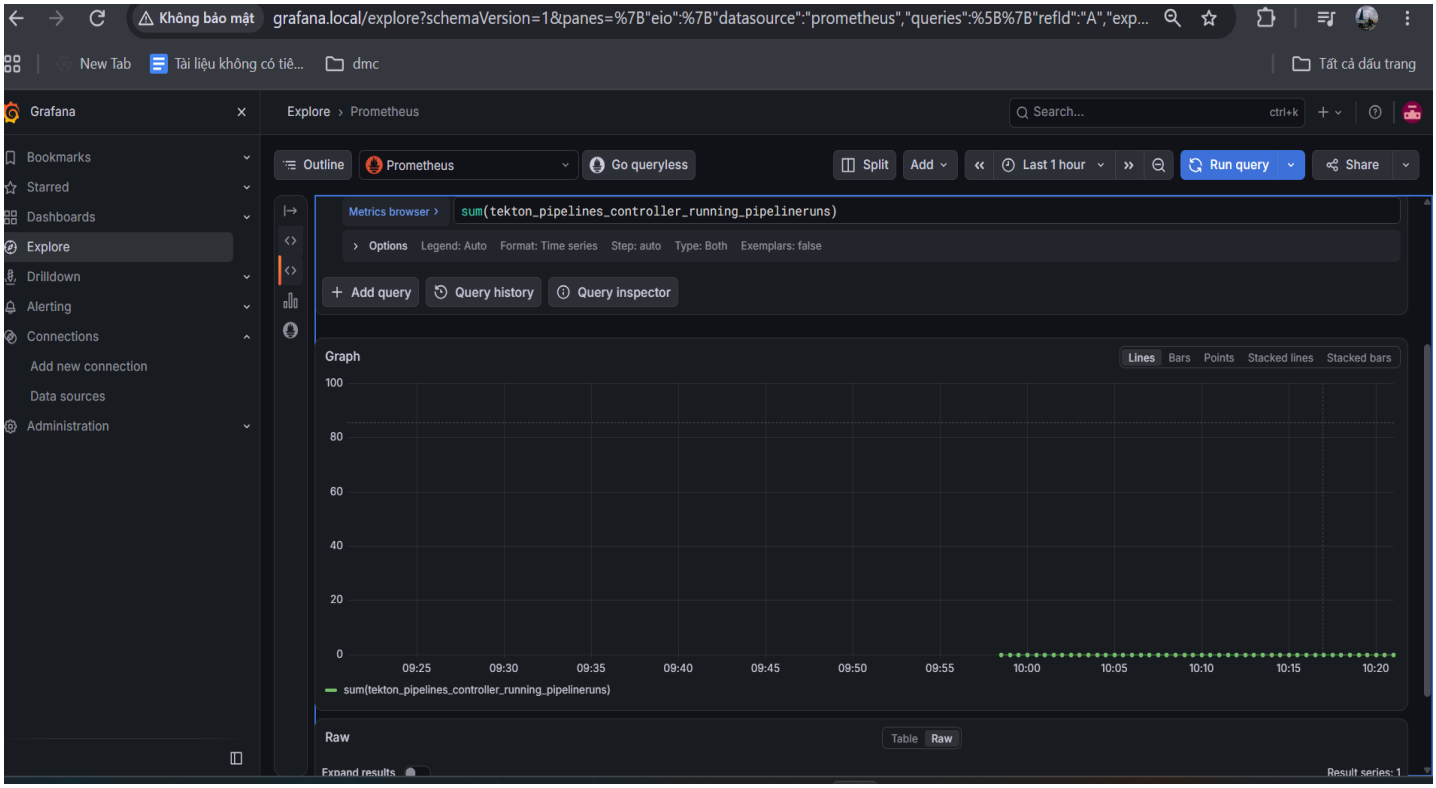
1. **Giám sát lưu lượng HTTP của Data Plane (ns-1 và ns-2):**

sum by (namespace, code)
(rate(promhttp_metric_handler_requests_total{job="demo-app-service"}[1m]))



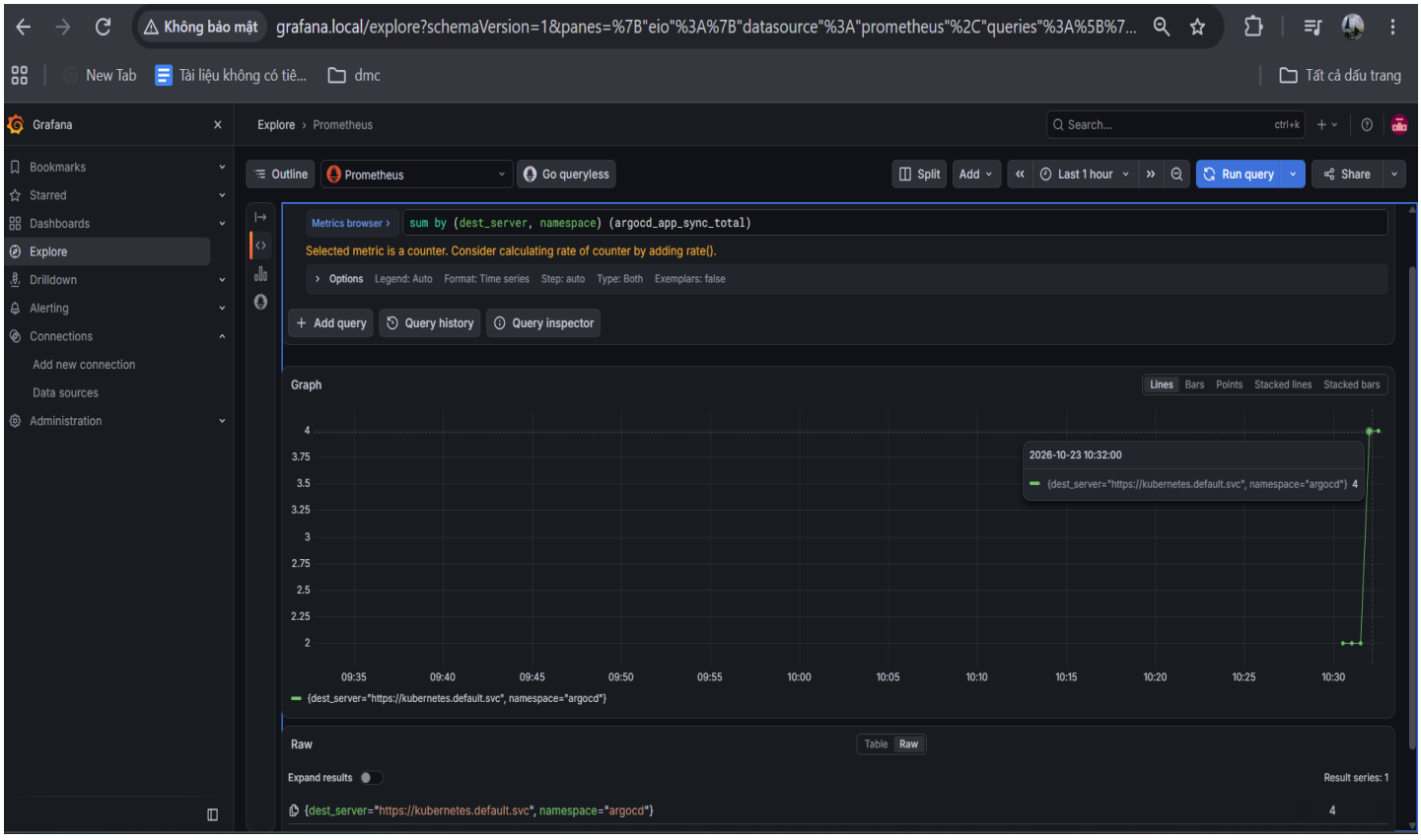
2. Giám sát số lượng PipelineRun của Tekton:

sum(tekton_pipelines_controller_running_pipelineruns)



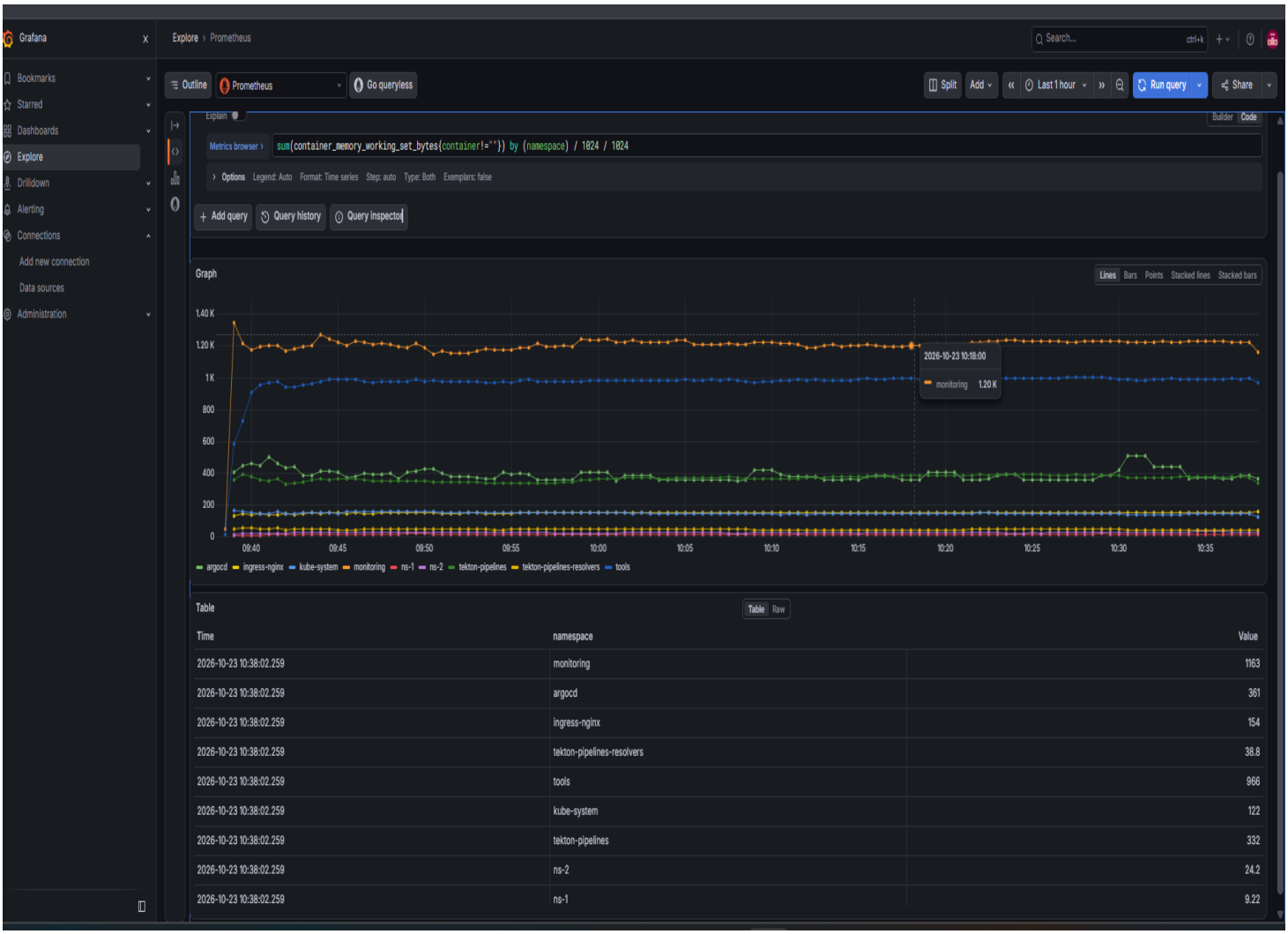
3. Giám sát trạng thái đồng bộ của Argo CD:

sum by (dest_server, namespace) (argocd_app_sync_total)

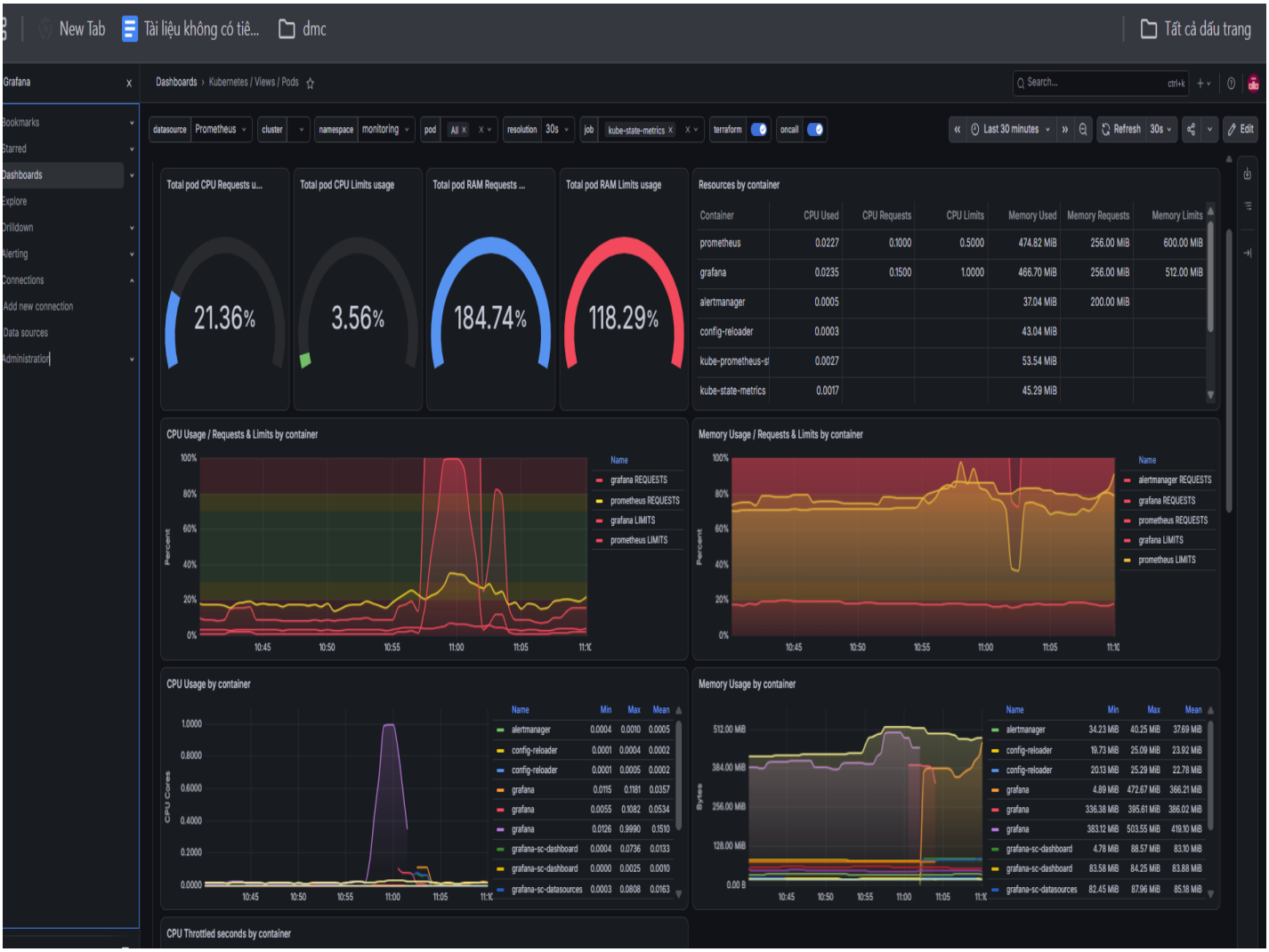


4. **Giám sát dung lượng RAM tiêu thụ (MB) của từng Namespace trên cụm K3d:**

sum(container_memory_working_set_bytes{container!=""}) by (namespace) / 1024 / 1024



Phần 3: Nạp Dashboard mẫu (Import Dashboard)



Tiêu chí đạt: Grafana tự động chuyển hướng vào Dashboard hoàn chỉnh với đầy đủ biểu đồ CPU, RAM, Network I/O và hệ thống file của cụm K3d đang cập nhật dữ liệu thời gian thực.